

Forex Fundamental x Technical Cross-Reference System

A browser-based app (plus the underlying scripts, if you prefer the command line) that:

1. Ranks currencies by fundamental strength — auto-fetched from FRED, uploaded, or edited right in the app.
2. Pulls price data and computes technical structure per pair (trend, momentum, volatility, swing structure).
3. Cross-references the two to surface which pairs currently have the clearest, most-aligned setups.

Quick start (the app)

1. Install Python from python.org if you don't have it.
2. **Mac:** double-click `run_app.command` (first time: right-click → Open, then click Open on the security prompt — macOS blocks unsigned scripts once by default). **Windows:** double-click `run_app.bat`.
3. It installs what it needs the first time (takes a minute), then opens the app in your browser automatically.
4. In the sidebar: pick a fundamental data source (FRED auto-fetch, upload a CSV, or just edit the built-in template), review/tweak the table, set your pairs, click **Run analysis**.

That's the whole workflow from then on — no terminal, no flags, no code.

Files

File	Purpose
<code>app.py</code>	The browser app — sidebar for data sources/settings, editable fundamental table, results tabs. Start it via the launcher, or <code>streamlit run app.py</code> .
<code>run_app.command</code> / <code>run_app.bat</code>	Double-click launchers (Mac / Windows).

<code>requirements.txt</code>	Packages the app needs (auto-installed by the launcher).
<code>fundamental_data_template.csv</code>	Starting fundamental data — either auto-fetched via FRED or filled in by hand.
<code>fred_fetch.py</code>	Auto-fetches fundamental data from FRED.
<code>fundamental.py</code>	Computes the currency strength ranking.
<code>technical.py</code>	Computes trend/momentum/volatility indicators and detects swing structure.
<code>data_feed.py</code>	Fetches OHLC price data (default: <code>yfinance</code> , no API key needed).
<code>engine.py</code>	Command-line entry point, if you'd rather script it than click through the app.

Command-line alternative

Everything in the app is also available as a script, if you prefer:

```

pip install -r requirements.txt
python engine.py --demo                # offline sanity check
python engine.py --fetch-fundamentals   # FRED auto-fetch + live run
python engine.py --fundamentals fundamental_data_template.csv # manual CSV

```

Setup

For the app: the launcher handles this automatically. For the command line:

```

pip install -r requirements.txt

```

1. Get your fundamental data

Option A — automatic (recommended): fetch it from [FRED](https://fredapi.stlouisfed.org/docs/api/api_key.html) (Federal Reserve Economic Data — free, no cost).

1. Get a free API key: https://fredapi.stlouisfed.org/docs/api/api_key.html (~2 minutes)
2. Set it: `export FRED_API_KEY=your_key_here`

3. Run: `python engine.py --fetch-fundamentals`

This writes/overwrites `fundamental_data_template.csv` with live data pulled from FRED, then runs the full analysis. **Open the CSV afterward to review it** — see “Coverage & honesty” below before trusting it blindly.

Option B — manual: edit `fundamental_data_template.csv` yourself (original workflow, still fully supported — just skip `--fetch-fundamentals`).

Columns either way:

- `InterestRate` — current policy rate (%)
- `RateBias` — forward-looking stance: 1 hawkish/hiking, 0 neutral/hold, -1 dovish/cutting (auto-fetch derives this from the recent rate trend)
- `CPI_YoY` — inflation, year-over-year (%)
- `GDP_QoQ` — GDP growth, quarter-over-quarter (%)
- `Unemployment` — unemployment rate (%)
- `PMI_Manufacturing`, `PMI_Services` — >50 = expansion, <50 = contraction (**not available via auto-fetch** — no reliable free API source exists; fill in manually if you want them included, otherwise leave blank and they’re excluded from that currency’s score rather than penalizing it)
- `TradeBalance_Bn` — trade balance (billions USD)
- `RetailSales_MoM` — retail sales, month-over-month (%)

Weights for each metric live at the top of `fundamental.py` (`WEIGHTS` dict) — tune them to reflect what you think matters most right now.

Coverage & honesty on the FRED auto-fetch

- **Verified, high-confidence series:** USD, EUR (interest rate + CPI), and interest rates for GBP/JPY/CAD — these are hardcoded to specific, checked FRED series IDs.
- **Everything else** (AUD, CHF, NZD, MXN, ZAR, TRY, SEK, and the remaining metrics for GBP/JPY/CAD) uses FRED’s series-search API at runtime to find the best match. This is genuinely best-effort — search ranking isn’t perfect, and a wrong or stale match is possible, especially for less-covered economies.
- Any metric it can’t find or fetch is left blank rather than guessed — check `fundamental_data_auto.csv` (or wherever `--fundamentals` points) after fetching, and hand-correct anything that looks off before trusting the ranking.

- If you have a paid data source (Trading Economics, Bloomberg, etc.), you can write your own fetcher — just produce a CSV in the same column format and skip `--fetch-fundamentals`.

2. Run it

```
# Offline demo (synthetic price data, no network/yfinance needed) -- sanity check
python engine.py --demo
```

```
# Auto-fetch fundamentals from FRED, then run live
python engine.py --fetch-fundamentals
```

```
# Manually-filled CSV, run live with default pairs (majors + a few common crosses
python engine.py --fundamentals fundamental_data_template.csv
```

```
# Live run with specific pairs
python engine.py --fundamentals fundamental_data_template.csv --pairs EURUSD,GBPU
```

```
# Different lookback/timeframe
python engine.py --period 1y --interval 1d
```

Output

1. **Currency strength ranking** — strongest to weakest, from your fundamental inputs.
2. **Per-pair table** — fundamental bias, technical score, detected structure (uptrend/downtrend/range), whether the two agree, and a conviction score.
3. **“Clearest market structure”** — the top pairs where fundamentals and technicals agree, ranked by conviction.

How scoring works (so you can trust/adjust it)

- **Fundamental strength:** each metric is z-scored *across the currencies you provide* (relative, not absolute), clipped at ± 2.5 std to stop one extreme outlier (e.g. a high-inflation currency) from dominating, then combined via the weights in `fundamental.py`.
- **Technical score:** 40% trend (EMA20/50/200 stack), 30% momentum (RSI + MACD histogram, normalized by ATR), 30% structure (swing high/low pattern: higher-highs+higher-lows = uptrend, lower-highs+lower-lows = downtrend, else range).
- **Conviction score:** `|fundamental bias| x |technical score|` when the two agree on direction; penalized ($\times 0.3$) when they disagree, rather than discarded — a disagreement

is itself informative (possible reversal or fundamentals not yet priced in).

Known limitations (read before trusting this for real decisions)

- **This is a screening/idea-generation tool, not a signal generator.** Forex is heavily driven by surprises (central bank decisions, geopolitical events, data releases) that no backward-looking model captures. Treat conviction scores as “worth a closer look,” not “trade this.”
- **You are responsible for fundamental data quality.** The system only reflects what you put in. Stale or wrong inputs produce misleading rankings.
- **The composite fundamental score is a simplification.** Real fundamental analysis involves reading central bank language, cross-asset flows, and positioning data — this reduces it to a weighted average, which is a real loss of nuance. Use it as a starting filter, not a replacement for judgment.
- **Swing structure detection uses a simple local-extremum method** (window=5 bars by default). It can misclassify structure in choppy conditions; sanity-check against a chart.
- **yfinance forex data has gaps and is delayed;** for anything time-sensitive, wire in your broker/data API instead (see `data_feed.py` — the interface is designed to be swapped out).

Where you said you want to extend this next

The architecture is modular on purpose so you can layer on:

- News/sentiment scoring feeding into fundamental bias
- Multi-timeframe technical confirmation (e.g. daily bias + 4H entry structure)
- Backtesting the conviction score against historical pair performance
- Alerting when a pair crosses a conviction threshold